

Node.js & NPM

Run JavaScript outside the browser. Learn Node, npm, modules, files and the terminal — then build four small projects, including your first web server.

WHAT WE COVER

Run JS anywhere

npm & modules

File system

CLI tools

HTTP server

4 projects

Until now your JavaScript lived inside a web page. Today it breaks free: Node.js runs JavaScript directly on your computer. This is a full session — we'll cover the core ideas with lots of small examples and short exercises, then build four mini-projects together, finishing with a real web server. Everything you need is in this handout.

Node.js & NPM

A browser runs JavaScript so pages feel alive. **Node.js** takes the same engine out of the browser so you can run JS on your machine — for tools, scripts and servers. We'll learn the tools with quick examples, do a few exercises, then build real projects.

CLASS AGENDA • ~2 HOURS

0:00	Warm-up: what Node is, and browser vs Node	§1–2
0:15	Run JS in the terminal + write your own modules	§3–4
0:35	NPM, npm init & understanding package.json	§5–8
0:55	System skills: arguments, files (fs), async & errors	§9–11
1:20	Build 4 mini-projects together	§12–15
1:55	Practice challenges & wrap-up	Challenges

ON THIS HANDOUT

- | | |
|---|---|
| 01. What is Node.js? | 09. Command-line arguments (argv) |
| 02. Browser vs Node | 10. The File System module (fs) |
| 03. Running JavaScript in the terminal | 11. Async in Node & error handling |
| 04. Modules: split your code | 12. Project A: Terminal Calculator |
| 05. What is NPM? | 13. Project B: Number Guessing Game |
| 06. Starting a project: npm init | 14. Project C: Word & File Counter |
| 07. Understanding package.json | 15. Project D: Your first Web Server |
| 08. Installing & using packages | |

PART 1 • Node.js Fundamentals

01 What is Node.js?

Your browser runs JavaScript using a built-in **engine** (Chrome's is called **V8**). **Node.js** takes that same V8 engine and turns it into a program you run on your computer — so JavaScript can run **outside** the browser.

- ▶ Node = the V8 engine + extra tools to work with files, networks and the system.
- ▶ It lets JS do what a web page can't: read/write files, run servers, build CLI tools.
- ▶ Same language you already know — brand new places to run it.

02 Browser vs Node

The language is identical; the surroundings differ.

- ▶ **Browser only:** the `document` / DOM, `window`, `alert`, clicking buttons.
- ▶ **Node only:** the file system, `process`, building servers, reading terminal input.
- ▶ **Both:** variables, functions, loops, arrays, objects, `console.log`, promises, async/await.

So there's no `document` in Node (no page), but you *do* get your files and terminal.

03 Running JavaScript in the terminal

First check Node is installed — open a terminal and ask for the versions:

```
node -v      # e.g. v22.x.x
npm -v       # e.g. 10.x.x
```

TERMINAL

Run a file

```
// hello.js
console.log("Hello from Node!");
console.log(2 + 3);
```

JS

```
node hello.js
# Hello from Node!
# 5
```

TERMINAL

The REPL — a quick playground

Type just `node` (no file) for an interactive prompt. Great for testing one-liners; type `.exit` to leave.

```
node
> 5 * 4
20
> "hi".toUpperCase()
'HI'
> .exit
```

TERMINAL

TRY IT YOURSELF

Create `hello.js` that prints your name and the current year using `new Date().getFullYear()`. Run it with `node hello.js`.

04 Modules: split your code

Real programs are split across files called **modules**. One file **exports** things; another **imports** them with `require`. This keeps code tidy and reusable.

Export from one file

```
// math.js
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }

module.exports = { add, multiply }; // share these two functions
```

JS

Import into another

```
// app.js
const math = require("./math"); // "./" = a file of ours (not a package)

console.log(math.add(2, 3)); // 5
console.log(math.multiply(4, 5)); // 20
```

JS

- ▶ `module.exports` decides what a file shares with others.
- ▶ `require("./math")` loads your own file; `require("fs")` (no `./`) loads a built-in module.
- ▶ Built-in modules come with Node — no install needed (we'll use several today).

TRY IT YOURSELF

Make `greetings.js` that exports a function `greet(name)` returning `"Hi " + name`. Import it in `app.js` and call it.

PART 2 · NPM & Project Setup

05 What is NPM?

npm (Node Package Manager) ships with Node and does two big jobs:

- ▶ **Manages your project** — tracks its name, version and settings in `package.json`.
- ▶ **Installs packages** — free, ready-made code from millions of libraries so you don't reinvent everything.

A **package** (or library) is reusable code you pull into your project with one command, downloaded from the npm registry.

06 Starting a project: npm init

Every Node project starts by creating a `package.json`.

```
mkdir calculator      # make a project folder
cd calculator         # go into it
npm init -y           # create package.json with default answers
```

TERMINAL

- ▶ `npm init` asks questions (name, version...); `npm init -y` accepts all defaults instantly.
- ▶ You'll now see a `package.json` — your project's ID card.

07 Understanding package.json

Open `package.json`. It's plain JSON describing your project:

```
{
  "name": "calculator",
  "version": "1.0.0",
  "description": "A simple terminal calculator",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "author": "Abhishek",
  "license": "MIT"
}
```

package.json

- ▶ `name` / `version` — the project's name and version number.
- ▶ `main` — the entry file that runs first.
- ▶ `scripts` — shortcuts run with npm, e.g. `npm start` runs `node index.js`.
- ▶ `dependencies` — added automatically when you install a package; lists what the project needs.

Two related files appear later: `node_modules` (the actual downloaded package code — never edited, never shared) and `package-lock.json` (records exact versions so everyone gets the same install).

Version numbers follow **semver** — `MAJOR.MINOR.PATCH` (e.g. `1.4.2`): patch = fixes, minor = new features, major = breaking changes.

08 Installing & using packages

Add a package with `npm install` (or `npm i`). Example — a fun one that prints big ASCII text:

```
npm install figlet
```

TERMINAL

- ▶ A `node_modules` folder appears with the downloaded code.

- `figlet` is added to `dependencies` in `package.json` so the project remembers it.

```
const figlet = require("figlet");
console.log(figlet.textSync("Hello!")); // big ASCII-art text
```

JS

TRY IT YOURSELF

Run `npm install figlet`, then write a script that prints a big banner with your name. Peek inside `package.json` to see it listed under dependencies.

PART 3 · Working With the System

09 Command-line arguments (argv)

You can pass values to a script on the command line. Node collects them in `process.argv` — an array where the first two entries are Node and the file path, so **your** arguments start at index 2.

```
// greet.js
const name = process.argv[2] || "stranger";
console.log(`Hello, ${name}!`);
```

JS

```
node greet.js Abhishek # Hello, Abhishek!
node greet.js          # Hello, stranger!
```

TERMINAL

This is how real command-line tools take input. Numbers arrive as text, so convert them with `Number(...)` when needed.

TRY IT YOURSELF

Write `add.js` so that `node add.js 4 7` prints `11`. (Hint: `Number(process.argv[2]) + Number(process.argv[3])`.)

10 The File System module (fs)

The built-in `fs` module lets JavaScript read and write files on your computer — something a browser can't do. Import it with `require`.

```
const fs = require("fs");

fs.writeFileSync("note.txt", "first line\n"); // create / overwrite
fs.appendFileSync("note.txt", "second line\n"); // add to the end

const content = fs.readFileSync("note.txt", "utf8"); // read it back
console.log(content);
// first line
// second line
```

JS

- ▶ `writeFileSync` creates or overwrites a file; `appendFileSync` adds to it.
- ▶ `readFileSync(path, "utf8")` returns the file's text (without `utf8` you get raw bytes).
- ▶ The `Sync` versions are simplest to learn; there are async versions too (next section).

TRY IT YOURSELF

Save a list of your three favourite foods to `foods.txt` (one per line), then read the file back and print it.

11 Async in Node & error handling

Reading files or calling the network can take time. Just like Day 5, Node handles this with **async** code — and you should wrap it in `try / catch` so a failure doesn't crash the program.

```
const fs = require("fs/promises"); // the promise-based version of fs

async function main() {
  try {
    await fs.writeFile("data.txt", "saved with async!\n");
    const text = await fs.readFile("data.txt", "utf8");
    console.log("read back:", text.trim());

    await fs.readFile("missing.txt", "utf8"); // this file doesn't exist -> throws
  } catch (err) {
    console.log("Caught an error:", err.code); // "ENOENT" = no such file
  }
}

main();
```

JS

- ▶ `fs/promises` gives you `await`-able file functions — cleaner than callbacks.
- ▶ `try / catch` catches problems (missing file, no permission) so you can respond gracefully.
- ▶ Every error has a `.message` and often a `.code` you can check.

TRY IT YOURSELF

Rewrite the `fs` read example from the previous section using `async/await`, and print a friendly message if the file is missing.

PART 4 • Build Four Projects

12 Project A: Terminal Calculator

Ask for two numbers and an operator, print the result, and loop until the user quits. Uses the built-in `readline` module to read typed input.

```
// index.js
const readline = require("readline");
const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
const ask = (q) => new Promise((resolve) => rl.question(q, resolve));

function calculate(a, op, b) {
  switch (op) {
    case "+": return a + b;
    case "-": return a - b;
    case "*": return a * b;
    case "/": return b !== 0 ? a / b : "Error: cannot divide by zero";
    default: return "Error: unknown operation";
  }
}

async function main() {
  console.log("=== Terminal Calculator ===");
  let again = true;
  while (again) {
    const a = Number(await ask("Enter first number: "));
    const op = await ask("Choose operation (+ - * /): ");
    const b = Number(await ask("Enter second number: "));
    console.log(`Result: ${a} ${op} ${b} = ${calculate(a, op, b)}\n`);
    const more = await ask("Calculate again? (y/n): ");
    again = more.toLowerCase() === "y";
  }
  console.log("Goodbye!");
  rl.close();
}
main();
```

Run it with `node index.js`. Note `Number(...)` — terminal input is text, so we convert it before doing maths.

13 Project B: Number Guessing Game

The computer picks a secret number 1–100; you guess until you get it, with higher/lower hints. Reinforces loops, conditionals and `Math.random()`.

JS

```
// guess.js
const readline = require("readline");
const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
const ask = (q) => new Promise((resolve) => rl.question(q, resolve));

async function main() {
  const secret = Math.floor(Math.random() * 100) + 1; // 1..100
  console.log("I'm thinking of a number between 1 and 100.");
  let guesses = 0;

  while (true) {
    const guess = Number(await ask("Your guess: "));
    guesses++;
    if (guess === secret) {
      console.log(`Correct! You got it in ${guesses} guesses.`);
      break; // leave the loop
    } else if (guess < secret) {
      console.log("Too low. Try higher.");
    } else {
      console.log("Too high. Try lower.");
    }
  }
  rl.close();
}
main();
```

- ▶ `Math.floor(Math.random() * 100) + 1` gives a whole number from 1 to 100.
- ▶ `break` ends the loop the moment the guess is correct.

14 Project C: Word & File Counter

A real command-line tool: pass it a file name and it reports the lines, words and characters. Combines `argv` + `fs` + error handling.

JS

```
// count.js
const fs = require("fs");

const filePath = process.argv[2]; // node count.js myfile.txt
if (!filePath) {
  console.log("Usage: node count.js <file>");
  process.exit(1); // stop, nothing to do
}

try {
  const text = fs.readFileSync(filePath, "utf8");
  const words = text.trim().split(/\s+/).filter(Boolean).length;
  const lines = text.split("\n").filter((l) => l.length > 0).length;

  console.log(`File: ${filePath}`);
  console.log(`Lines: ${lines}`);
  console.log(`Words: ${words}`);
  console.log(`Characters: ${text.length}`);
} catch (err) {
  console.log("Could not read file:", err.message);
}
```

TERMINAL

```
node count.js sample.txt
# File: sample.txt
# Lines: 2
# Words: 9
# Characters: 44
```

`split(/\s+/)` breaks the text on any run of spaces/newlines; `filter(Boolean)` drops empty pieces so the count is accurate.

15 Project D: Your first Web Server

The big one: Node's built-in `http` module lets you build a real web server in a few lines. This is the same JavaScript acting as a **backend**.

JS

```
// server.js
const http = require("http");

const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Hello from my Node server!");
  } else if (req.url === "/time") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ time: new Date().toLocaleTimeString() }));
  } else {
    res.writeHead(404);
    res.end("Page not found");
  }
});

server.listen(3000, () => console.log("Server running at http://localhost:3000"));
```

- ▶ `createServer` takes a function that runs on every request (`req`) and sends a reply (`res`).
- ▶ `req.url` lets you respond differently per address — that's basic **routing**.
- ▶ `server.listen(3000)` keeps the program running and waiting for visitors.

Run `node server.js`, then open `http://localhost:3000` in your browser. Visit `/time` too. Stop the server with Ctrl+C.

Practice Challenges

Pick a few to try in class or as homework — they reuse everything above:

- ▶ **Temperature converter (CLI):** `node temp.js 100 C` → prints Fahrenheit.
- ▶ **Dice roller:** print a random 1–6, and keep rolling while the user types `y`.
- ▶ **Notes app:** `node note.js "buy milk"` appends the note to `notes.txt`; `node note.js` lists them.
- ▶ **Extend the server:** add an `/about` route that returns your name.
- ▶ **Improve the calculator:** add power (`**`) and remainder (`%`), and reject non-numbers.

Conclusion & what's next

That's a lot in one session. You learned that **Node.js** runs JavaScript on your computer, how to split code into **modules**, how **npm** and **package.json** manage a project, how to read command-line **arguments** and **files**, how to handle **async** work safely — and you built four programs, ending with a real web server.

Where this goes next: the http server you just wrote is the seed of every backend. Next we'll use a framework called **Express** to build proper APIs — and connect them to the front-end apps you've already made.

QUICK RECAP

Node runs JS outside the browser · **modules** split code with require/exports · **npm** + **package.json** manage projects · **argv** reads input, **fs** reads/writes files · and the **http** module makes JavaScript a real backend.